ChatGPT

Reasoning in LLMs: Prompting, Test-Time Compute, and Process Supervision

1. Introduction: Motivation and Challenges

Large Language Models (LLMs) have shown remarkable capabilities, yet they often struggle with complex reasoning tasks that require multiple steps or strategic planning [1] [2]. Standard prompting (directly mapping an input to an output) tends to break down on tasks like math word problems, logical reasoning, or code synthesis. This is because naive **end-to-end** generation encourages the model to guess an answer without a disciplined reasoning process. As a result, even very large models can produce incorrect or inconsistent answers if they don't "think through" the problem [3] [4].

A key insight from recent research is that **allocating more computation at inference time** – in effect, letting the model reason longer or explore multiple solution paths – can substantially improve accuracy on challenging tasks [5] [6]. This approach is appealing because it leverages *test-time compute* (which is often cheaper than retraining a larger model) to enhance reasoning, rather than solely relying on scaling model size. Indeed, OpenAI's new *o1* model is explicitly designed to "spend more time thinking before responding," using internal chains-of-thought and iterative refinement [7] [5]. Empirically, o1 achieves breakthroughs on competition-level math and science problems by combining large-scale training with extended inference-time reasoning [8] [6].

Another motivation is to improve the **reliability and interpretability** of model reasoning. When models can articulate intermediate steps (a "chain of thought"), it provides a window into their problem-solving process, making it easier to diagnose errors [9] [10]. Moreover, supervising the *process* (each reasoning step) rather than just the final answer can lead to more robust performance [11] [12]. **Process supervision** means providing feedback at intermediate steps, which encourages the model to avoid logical mistakes and hallucinations in the middle of a solution, not just at the end [13] [11]. This has been shown to train more reliable problem-solvers – for example, a process-supervised model solved 78% of problems in a challenging math benchmark, far outperforming outcome-supervised counterparts [11]. Additionally, process feedback can guide *open-ended trajectories* at inference: the model can be nudged step-by-step, much like a human tutor guiding a student through a difficult problem, to explore solution paths without immediately converging on a wrong answer.

In the rest of this guide, we will explore cutting-edge techniques for improving reasoning in LLMs, blending theoretical foundations with practical examples and exercises. We begin with **Chain-of-Thought prompting** (which lets an LLM generate explicit reasoning steps) and **self-consistency decoding** (which uses multiple reasoning samples to boost reliability). We then discuss the **ReAct paradigm** that interleaves reasoning with actions (allowing an LLM to query external tools or environments), and how **process supervision** can be applied to reinforce each reasoning step. Next, we examine **Tree-of-Thoughts** and other search-based inference methods that enable exploring many reasoning branches. We will also review the recent **OpenAI o1 (2024)** model and **DeepSeek-R1 (2025)**, which showcase advanced multi-path and multi-agent reasoning techniques. After covering strategies to **scale test-time compute** – such as routing, depth sampling,

consistency filtering, and beam re-ranking – we provide a hands-on **prompt-engineering lab** with exercises on CoT, ReAct, and Tree-of-Thoughts prompting. Finally, we survey benchmark results to quantify the trade-offs between reasoning accuracy and inference compute, and conclude with key insights and open challenges in this rapidly evolving domain.

2. Chain-of-Thought Prompting and Self-Consistency

One of the most influential ideas for improving reasoning is **Chain-of-Thought (CoT) prompting** [14]. Instead of asking the model to directly output an answer, we prompt it to produce a *series of intermediate steps* – a natural language reasoning chain – that leads to the final answer [15] [16]. For example, consider a multi-step math word problem. Rather than responding with a single number, a CoT-enabled model will first outline the solution path: "*Roger started with 5 balls. He buys 2 cans of 3 balls each (that's 6 balls), so now 5+6=11. …*" and so on, before concluding with the answer. By decomposing problems into steps, the model can tackle each part incrementally, reducing errors and allowing more complex reasoning than a one-shot response [17] [9].

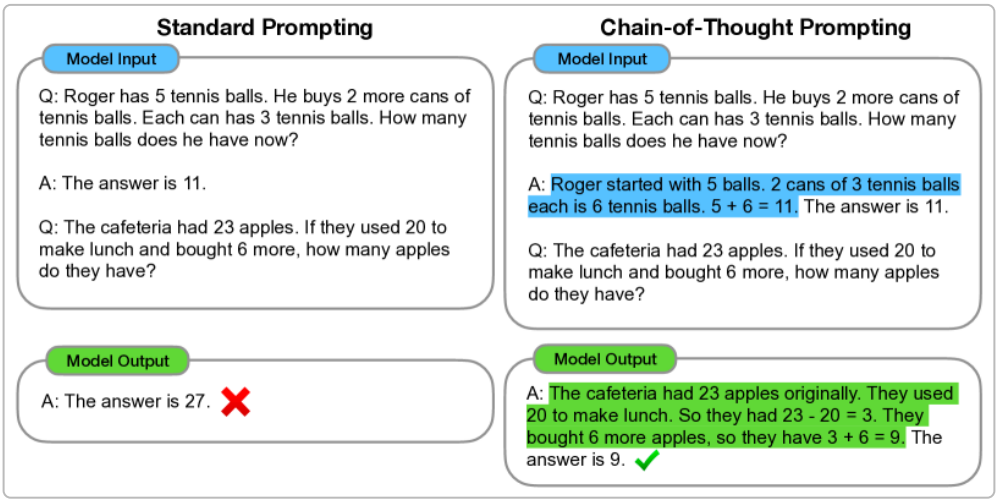


Figure: Example of standard prompting versus chain-of-thought prompting on arithmetic word problems. The *Standard Prompting* approach (left) directly outputs an answer ("27") which is incorrect. In contrast, the *Chain-of-Thought Prompting* approach (right) has the model explicitly reason through the problem (*green text*), computing intermediate results step by step, and arrives at the correct answer ("9") [9] [18]. CoT prompts thus encourage the model to "think aloud," yielding interpretable solutions and often higher accuracy.

CoT prompting was introduced by Wei *et al.* (2022) [14], who demonstrated striking improvements across arithmetic, commonsense, and symbolic reasoning tasks. Notably, a 540B-parameter model (PaLM) with CoT few-shot prompting achieved *state-of-the-art* on the GSM8K math word problem benchmark, surpassing even a finely-tuned 175B model that had a complex verifier in its pipeline [1] [19]. These gains highlight an important point: **CoT effectively allows a model to use more inference steps for harder problems** [18]. In a sense, the model can allocate extra "mental computation" to a challenge by writing out a longer solution – something not possible when forced to answer in one step. CoT also scales with model size: sufficient scale is needed for models to generate coherent, useful chains of thought [9]. Below a certain parameter threshold, models might produce irrelevant or erroneous chains; but beyond that (in the original

paper, ~100B+ parameters), the ability to follow multi-step prompts "emerges" and improves with size [14] [20].

2.1 CoT Prompt Design

To use CoT prompting in practice, one typically provides a few **exemplars** (few-shot examples) in the prompt demonstrating the format: each exemplar consists of a query, a detailed reasoning chain, and a final answer [21]. For instance, a prompt for a math problem might look like:

```
Q: John has 2 cats and each cat has 4 kittens. How many kittens in total?
Chain-of-thought: Each cat has 4 kittens, and there are 2 cats, so 2 * 4 = 8.
Answer: 8

Q: Sarah bought 3 boxes of eggs, each with 12 eggs, then used 5 eggs for
cooking. How many eggs remain?
Chain-of-thought: First calculate total eggs: 3 boxes * 12 = 36. After using 5,
remaining = 36 - 5 = 31.
Answer: 31

Q: *<new question>* ...
Chain-of-thought:
```

By giving a few solved examples with their reasoning, we prime the model to continue the pattern for new queries. Importantly, we include the literal phrase "Chain-of-thought:" (or a similar cue like "Let's think step by step") to signal the model to produce a rationale [21] [9]. This simple approach guides the model into a reasoning mode. Research has found even zero-shot CoT can be effective if you prompt with a phrase like *"Let's think step by step"* before the answer, exploiting the model's learned heuristic that this phrase often precedes a logical explanation [22] [23].

CoT has several **benefits**. First, it breaks complex problems into simpler intermediate steps, as noted above [18] [24]. Second, it provides interpretability – we can inspect the chain-of-thought to understand the model's logic and identify where it might have gone wrong [25] [10]. This is invaluable for debugging model errors or for users to gain trust in the answer. Third, CoT is quite general: it works for math, logic puzzles, commonsense QA, symbolic reasoning, and more [26] [27]. In principle, any task a human could solve via stepwise reasoning can be given to a sufficiently advanced LLM with CoT. Finally, CoT doesn't require fine-tuning – it's an *in-context learning* technique, meaning the base model remains unchanged and we only craft the prompt. This makes it very convenient to apply across tasks without new training data [28].

However, CoT prompting alone is not a panacea. A model might generate a long chain-of-thought that **looks** plausible but still arrives at a wrong answer (so-called *illusory reasoning*). It might also include unnecessary tangents or get stuck in loops. To address some of these issues, researchers have developed improved decoding strategies on top of CoT, the most prominent being **self-consistency decoding**.

2.2 Self-Consistency Decoding

Self-Consistency (SC) is a decoding approach that leverages the observation that a reasoning problem can often be solved via multiple distinct valid chains of thought, all leading to the same correct answer [29] [30]. Instead of relying on a single chain-of-thought (which might be flawed or just one of many possible solution paths), we sample *multiple* reasoning chains and then aggregate their answers to find a consistent result [29] [31]. In practice, this means using *stochastic decoding* (like sampling or temperature > 0) to generate, say, 5 or 10 different CoT outputs from the model for the same question. Each of those chains yields a final answer. The **self-consistency** step then selects the answer that appears most frequently among the candidate chains, under the intuition that the correct answer will be consistently reached by different reasoning paths, whereas wrong answers will be more randomly distributed [31] [32].

This approach, proposed by Wang *et al.* (2022) [33], has shown *striking gains* in performance. For example, on the GSM8K math dataset, self-consistency boosted CoT accuracy by ~18 percentage points (from around 40% to 58% solved) [34]. Similar double-digit improvements were seen on other benchmarks like SVAMP (+11%), AQuA (+12%), and significant gains on commonsense reasoning tasks [34]. Essentially, SC combines the strength of *diverse reasoning* with a simple form of ensemble voting. It is a pure inference-time method – no additional training needed beyond the base CoT capability.

Why does self-consistency work so well? One reason is that it mitigates the "one bad chain" problem. If your single CoT sample goes astray due to a minor error, the final answer is wrong even if the model *could* have solved it with a different thought process. By drawing multiple samples, you increase the chance that at least one chain is correct. The voting then amplifies the signal of the correct answer if multiple chains converge on it. Another reason is that it encourages **diversity in reasoning**: one chain might use algebra to solve a problem, another might simulate with numbers – if both agree on the answer, that's strong evidence the answer is correct. On the other hand, if the chains yield different answers, the model (or a user) can recognize uncertainty and possibly trigger further analysis or fallback strategies.

From an implementation standpoint, self-consistency is straightforward. In pseudocode, one might do:

```
answers = []
for i in range(N):
    reasoning = model.generate(question, prompt_mode="chain-of-thought")
    final_ans = extract_answer(reasoning)
    answers.append(final_ans)
# Select the most frequent answer (majority vote)
best_answer = max(set(answers), key=answers.count)
```

Typically N could be 5, 10, or even more (OpenAI's o1 model was often evaluated with 16 or 64 samples for high-stakes tasks [8] [6]). There is a trade-off: more samples improve the odds of finding the correct answer (and consensus) at the cost of higher latency and computation. In research settings, using 100+ samples is not uncommon to maximize accuracy [8], but in practical deployments one might use fewer or make it adaptive (e.g., stop early if a clear majority emerges among the first few samples).

Self-consistency connects to the idea of **model confidence** as well. Rather than trust a single model's probability or self-estimated confidence, we ask the model to *generate many possible solutions*. If it truly

“knows” the answer, most of those sampled solutions – even if phrased differently – will reach the same conclusion. If it’s uncertain or the question is ambiguous, the answers will scatter. This approach has been found more reliable than a raw probability score of one answer, especially for reasoning tasks [35] [36]. In essence, majority voting among model outputs serves as a form of uncertainty quantification.

It’s worth noting that to make self-consistency most effective, the reasoning samples should be **diverse**. Simply sampling with a low temperature might give very similar chains, which defeats the purpose. Techniques like using higher sampling temperature or adding slight prompt variations can help cover different reasoning paths. Some works also explore weighting votes by the model’s log-probability of each answer or using a separate verifier model to rank the answers instead of pure majority count [37] [6]. But the simplest majority vote has proven remarkably strong.

In summary, Chain-of-Thought prompting unlocks latent reasoning capabilities of LLMs by making them break down problems, and self-consistency further boosts reliability by ensembling multiple reasoning attempts. These techniques significantly narrow the performance gap on tasks that were once thought to require explicit training or symbolic tools. Still, CoT and self-consistency operate *entirely within the model’s own mind*. Next, we explore approaches that let the model **interact with external tools or environments** to aid reasoning, and how supervising the reasoning *process* can address some limitations of pure CoT.

3. ReAct: Integrating Reasoning and Acting (and Process Supervision)

While chain-of-thought prompting lets an LLM reason internally, it doesn’t allow the model to **interact with the outside world** during that reasoning. Humans solving problems often consult external resources (books, tools, environment feedback) as part of their thinking process. The **ReAct** framework, introduced by Yao et al. (2023) [38] [39], combines “Reasoning + Acting” in prompting, enabling language agents that can both *think* (generate thoughts) and *act* (take actions that affect or query an external environment). In ReAct, the model’s output is structured to alternate between **Thought** (natural language reasoning steps) and **Action** (commands or queries to an external tool/environment), with corresponding **Observations** returned to the model [40] [41].

Concretely, a ReAct prompt might look like this for a QA task with a web search tool:

```
Question: "Who was president of France in 2015 and what is their current age?"
Thought 1: I should identify who was President of France in 2015.
Action 1: Search["President of France 2015"]
Observation 1: The President of France in 2015 was François Hollande (born
1954).
Thought 2: Hollande was born in 1954, and now it's 2025, so he is 71 years
old.
Action 2: Finish["The president in 2015 was François Hollande, and he is 71
years old."]
```

Here, the model reasons in Thought 1 about how to approach the query, then issues Action 1 to search the web. The environment (a search API) returns an Observation 1 (Hollande’s info). The model then continues

reasoning with that info in Thought 2, and finally produces a formatted answer via a Finish action. This interleaving allows dynamic planning: the model can gather evidence or perform intermediate computations as needed, rather than relying solely on its frozen internal knowledge [42] [43].

Why ReAct? The original paper demonstrated several advantages [39] [44]:

- ReAct *outperformed* both reasoning-only agents (which can get stuck hallucinating facts) and acting-only agents (which blindly perform actions without high-level deliberation) on diverse tasks [39]. It achieved state-of-the-art results in open-domain QA (HotpotQA, FEVER) by effectively using Wikipedia search during multi-hop reasoning [45]. It also excelled in interactive tasks like an ALFWorld text-based game and a WebShop navigation task, where pure chain-of-thought wasn't enough to find the answer [46]. For example, on the ALFWorld game, a ReAct agent, prompted with just a couple of examples, outperformed specialized reinforcement learning policies by a large margin [46].

- ReAct yields **more interpretable and diagnosable** solutions. By reading the chain-of-thought and the sequence of actions, a human can follow exactly what the agent tried and where it might have failed [44] [47]. This is useful for debugging or for safety – we can intervene if we see the agent about to take a wrong action. It's a step toward transparent AI systems that can explain how they arrived at an answer by showing the intermediate steps and sources.

- Combining reasoning with acting enables a form of **grounding and feedback**. The model's thoughts are no longer a "static hallucination" – they are continually checked or informed by observations from the environment [48] [49]. If the model has a faulty assumption, an external query might correct it (e.g. a calculator tool would prevent arithmetic slips in its reasoning chain). This tight loop can reduce hallucinations and error propagation, as the model can correct itself when an action's result contradicts its prior belief [48] [50].

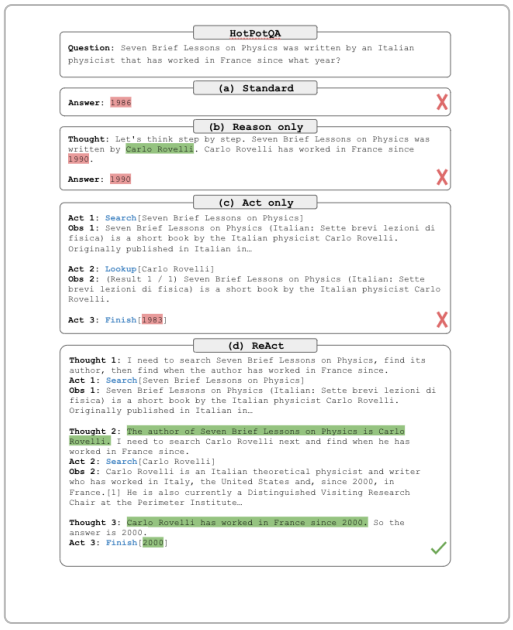


Figure: Illustration of ReAct prompting versus ablations on a sample task (HotpotQA). *(a)* **Standard** prompting directly outputs an answer with no reasoning (here incorrect). *(b)* **Reason Only** prompting

generates a chain-of-thought but without acting (the model tries to recall facts from memory; it correctly identifies the person but gives the wrong year, 1990). *(c)* **Act Only** prompting uses tool actions without explicit reasoning (the model searches and finds some info but fails to synthesize it correctly, yielding "1983"). *(d)* **ReAct (Reason + Act)** combines both – the model's thoughts plan a strategy: first search for the book's author, then find the year he worked in France. The actions retrieve relevant facts (Carlo Rovelli's details), and the reasoning uses them to arrive at the correct answer "2000". ReAct's synergy is evident: it can retrieve up-to-date or obscure information (via actions) and reason about it coherently (via thoughts), solving tasks neither approach alone can handle.

Implementing ReAct systems involves defining the **action space** and ensuring the LLM understands the syntax. In many cases, actions are text-based API calls, like `Search[...]`, `Lookup[...]`, `Calculate[...]`, or domain-specific moves in a game environment. Few-shot examples in the prompt demonstrate how to format these actions and how observations appear. The LLM then generates outputs in that format. A loop is set up to alternate between model generation and environment response until a terminal action (like `Finish[...]`) is produced.

From a coding perspective, a simplified ReAct loop might look like:

```
history = []
obs = None
while True:
    # Construct prompt from history and latest observation
    prompt = format_prompt(question, history, obs)
    output = model.generate(prompt)
    thought, action = parse_react_output(output)
    history.append((thought, action))
    if action.name == "Finish":
        answer = action.argument  # the final answer string
        break
    else:
        obs = env.execute(action)  # perform the action and get observation
```

This pseudo-code highlights that the LLM's output `output` contains both a thought and an action. We parse it, log the thought (for interpretability), perform the action in the environment (`env.execute` could be a web search API, a calculator, etc.), and loop. The loop ends when the model outputs a special `Finish` action (with an answer).

3.1 Process Supervision in ReAct and Beyond

ReAct provides a framework for *process-level* intervention: because the reasoning process is exposed step-by-step, we can potentially **evaluate and supervise each step**, not just the final answer. This is where **process supervision** comes in. While the example above used the environment's feedback as a kind of supervision (the model adjusted its strategy after the search results), we can also apply *learning* signals at the reasoning step level. For instance, if an LLM proposes a thought or action that is clearly incorrect (e.g. a wrong calculation or a search query that doesn't make sense), a process supervisor can correct or penalize that immediately, rather than only giving a reward at the end if the final answer is wrong [51] [52].

OpenAI's experiments have shown that training LLMs with process supervision (using human or AI feedback on each intermediate step) can significantly improve performance on complex reasoning. In the domain of math problems, **process-supervised models** learned to avoid common errors by receiving feedback at each step of their derivations [11]. Compared to outcome-supervised models (which only get a signal after producing a final answer), process-supervised models are easier to align with human-like solution strategies and are *more interpretable by design* [53] [13]. Furthermore, process supervision has benefits for safety and alignment: it's easier to detect where a reasoning process goes off-track (e.g., starts using a forbidden tool or following a biased assumption) and correct it, than to untangle *why* a final answer is problematic if you only judge the outcome.

In reinforcement learning terms, process supervision can be implemented via a **reward model (RM)** that scores each intermediate thought/action. Lightman *et al.* (2023) trained a Process Reward Model on thousands of human-labeled reasoning steps [54]. This PRM could then be used to guide an RL-trained LLM to produce better step-by-step solutions, achieving the notable 78% solve rate on MATH problems mentioned earlier [11]. Other research has proposed enhancing these reward models with lookahead capabilities (predicting not just if a step is correct, but if it's *leading toward* a correct final answer) [51] [55], akin to how A *search uses both backward and forward cost estimates. This Bi-directional process supervision** (evaluating past steps and future promise) further improves the guidance provided to LLMs during reasoning, especially in search-based inference settings [56] [57].

It's important to note that process supervision can be very resource-intensive: OpenAI's PRM was trained on ~800k step annotations [58], a significant data collection effort. However, there are some semi-automated ways to gather process feedback: one can use *self-critiquing* models (have the model judge its own steps or another model's steps), or use simulation (generate synthetic reasoning paths and label them via consistency checks or known logical rules). A promising direction is **active learning** – Lightman *et al.* report a 2.6× improvement in data efficiency by cleverly selecting which steps to label next [54]. In practice, process supervision might involve a mix of automated checks (e.g., verifying a calculation with a calculator tool as the step is generated) and learned feedback.

3.2 Open-Ended Trajectory Guidance

"Open-ended trajectory" refers to a reasoning process that isn't fixed in length or structure beforehand – the model can take as many steps as needed, possibly exploring multiple avenues, until it finds a solution. Both CoT and ReAct allow some degree of open-endedness (especially ReAct, where the agent can keep acting in a loop). However, guiding a model through a long, branching problem-solving **trajectory** is challenging. Issues that arise include: how to know when to stop reasoning, how to backtrack if a line of thought fails, and how to avoid the model getting "lost" in an endless chain of steps.

One technique to guide open-ended reasoning is to employ **self-reflection**. After a series of steps, the model can be prompted to reflect: *"Have I made a mistake or a false assumption? Should I try a different approach?"*. Shinn *et al.* (2023) introduced a method called "Reflexion" where the model, upon reaching a dead-end or incorrect answer, is given its own reasoning log and asked to critique it and attempt a new solution [59]. This often leads to significantly better results through iterative refinement. In essence, the model is acting as its own coach, adjusting its trajectory based on past mistakes – a form of inner-monologue debugging.

Another approach is to incorporate a **planning heuristic or controller** that monitors the trajectory. For example, in search-based methods (like the Tree-of-Thoughts we discuss next), one can use a value function to estimate progress towards the goal and prune or prioritize certain branches [60] [61]. Similarly, in an open-ended ReAct agent, one could impose rules like "if you haven't made any new progress in 5 steps, reconsider your approach" or have a secondary model oversee and suggest next steps (a kind of meta-reasoner). This resembles a multi-agent setup, where one agent is the "solver" and another is the "guide" or "critic". Indeed, some recent works use multiple LLM agents in a loop – e.g., one generates a solution, another checks it, and they converse until consensus is reached [62] [63]. Such **multi-agent reasoning** can be seen as the model debating or double-checking with itself to avoid premature or wrong answers.

In summary, integrating acting with reasoning (ReAct) and supervising the reasoning process stepwise are powerful strategies to enhance LLM problem-solving. They transform the task from a one-shot guess to an interactive, deliberative journey. The model can gather evidence, correct itself, and be guided at each step, leading to more accurate and trustworthy outcomes. However, ReAct and stepwise supervision also introduce more complexity – we need to manage action protocols, handle errors in the middle of a chain, and possibly deal with higher computational cost due to multiple steps. Next, we delve into a framework that generalizes the idea of exploring multiple reasoning paths: the **Tree-of-Thoughts**, which will naturally connect with many concepts discussed here, such as search-based inference and trajectory guidance.

4. Tree-of-Thoughts and Search-Based Inference

The **Tree-of-Thoughts (ToT)** paradigm extends chain-of-thought by allowing the model to **explore different branches of reasoning** rather than committing to a single line of thought in a left-to-right sequence [64] [65]. This idea, proposed by Yao et al. (2023) [66], is inspired by how humans solve complex problems: we may brainstorm multiple approaches, consider intermediate results, backtrack from a dead end, and choose the most promising path to follow. Instead of a linear chain, ToT maintains a *tree* of possible thoughts, where each node is an intermediate reasoning state, and branches represent alternative next steps or ideas [67] [68].

In a Tree-of-Thoughts, the model generates **multiple candidate thoughts at each step**, then evaluates them (either with a heuristic or by looking ahead a few steps) to decide which branches to expand further [69] [70]. This process continues until a solution is found or a depth limit is reached, at which point the best solution among the explored branches is selected. The approach brings classical AI search algorithms into play (like breadth-first search, depth-first search, or even Monte Carlo Tree Search) but uses the language model as both a *generator* of next-step options and as an *evaluator* of partial solutions.

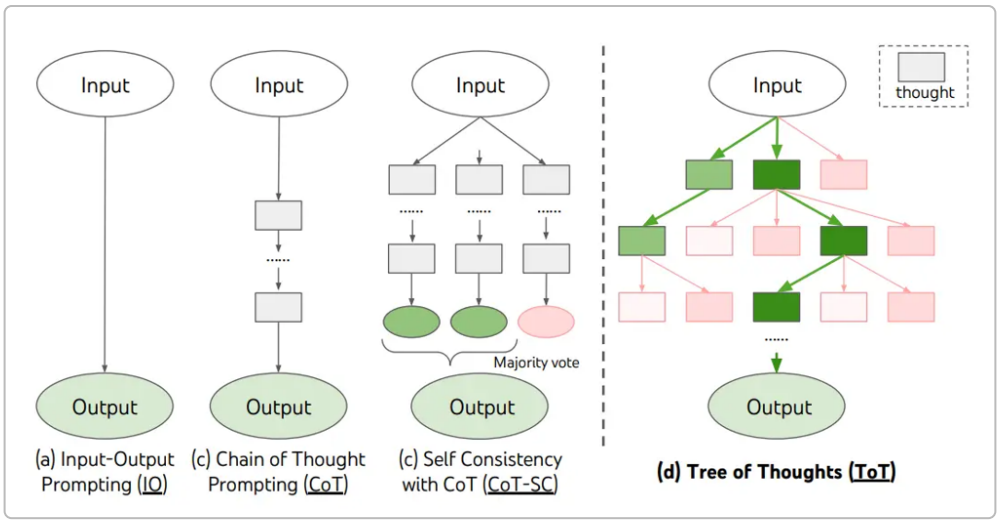


Figure: Comparison of prompting strategies and the Tree-of-Thoughts framework [71]. *(a)* Standard prompting (direct Input → Output) uses no intermediate reasoning. *(b)* Chain-of-Thought (CoT) prompting introduces a linear sequence of thoughts leading to one output. *(c)* Self-Consistency with CoT samples multiple independent chains and then chooses the answer by majority vote [31]. *(d)* Tree-of-Thoughts (ToT) explicitly builds a branching structure of thoughts, exploring several reasoning paths in parallel and converging to an output after evaluating the branches. Green boxes indicate promising thoughts that are pursued, while red boxes indicate pruned or less likely paths (evaluation might label thoughts as "sure", "maybe", or "impossible" towards solving the problem) [60] [61]. This diagram highlights how ToT generalizes CoT: instead of one linear thought sequence, we have a search tree over possible reasoning steps.

The ToT approach shines on problems that require **strategic exploration or planning** where a wrong early decision can lead one far off course [64] [72]. For instance, consider a puzzles like *"24 Game"*: given four numbers, find a way to combine them with operations to make 24. A chain-of-thought might pursue one combination and fail, whereas Tree-of-Thoughts can try different operation orders systematically. In the original paper, ToT with GPT-4 solved 74% of 24-Game puzzles, compared to only 4% solved by standard CoT with GPT-4 [73] [74] – a dramatic improvement showing the power of search-based inference. Other tasks demonstrated included creative writing (where the model can explore different story continuations and pick the best) and mini crosswords, both requiring lookahead and sometimes backtracking [72].

A typical Tree-of-Thoughts algorithm might work as follows (illustrative pseudo-code for a BFS variant):

```
state0 = initial_state(problem)  # no thoughts yet
frontier = [ state0 ]

for depth in range(max_steps):
    new_frontier = []
    for state in frontier:
        thoughts = model.generate_n_candidates(state, n=B)  # propose B next-
step thoughts
        for th in thoughts:
            new_state = state + [th]  # append thought to the chain
```

```
            score = model.evaluate_state(new_state, problem)
            if score == "sure" or depth == max_steps-1 and score !=
"impossible":
                # If the model is confident this leads to a solution, or we've
reached final depth
                solutions.append(new_state)
            if score == "maybe":
                new_frontier.append(new_state)  # continue expanding this branch
    frontier = select_top_k(new_frontier, k=K)  # prune to top K most promising
states
```

In this sketch, at each depth the model generates up to **B** candidate next thoughts for each current reasoning state. The model (or a heuristic) evaluates each new partial reasoning chain as "sure, maybe, or impossible" (this was the scheme used by Yao et al. [60]). "Sure" means the chain seems to be on a direct path to a correct solution (high confidence), "impossible" means it cannot lead to a solution (perhaps a contradiction or running out of steps), and "maybe" is intermediate. Only "maybe" (and implicitly "sure") paths are kept for further expansion. Additionally, to keep the search manageable, one may keep only the top **K** states at each depth (beam search style), based on some priority – e.g., the model's estimated probability that the partial solution can reach the goal.

The above process blends the model's generative ability with a **search algorithm**. Notably, the model is used in two roles: (1) proposing possible thoughts, and (2) evaluating partial progress. The evaluation could be done by the same model prompted to judge "Is this line of reasoning promising?" or by a separate value model. In the absence of a trained value function, one can prompt the model to explicitly assign a verdict like in the example: *"Rate this partial solution as [sure / maybe / impossible] in terms of reaching the goal."* [60] [61]. This leverages the model's own knowledge (and hopefully some common sense or logical estimation) to prune the search space.

A concrete example from the Tree-of-Thoughts paper is the **Game of 24** (make 24 with given numbers). The solution requires exactly 3 steps (combining four numbers into one result). The ToT approach set B=5 (generate 5 candidate equations at each step) and then had the model evaluate each partial equation as definitely leading to 24, possibly leading to 24, or impossible (e.g., if partial result already too large or fractional, etc.) [75]. By doing a breadth-first search with these heuristics, ToT was able to eliminate hopeless branches early and find the correct solution in a much larger fraction of cases than a linear attempt. The key was that the model's *own heuristic judgment* was used to navigate the huge space of possible equations – effectively a learned heuristic function guiding a systematic search.

From a performance standpoint, Tree-of-Thoughts can significantly boost success rates on reasoning tasks, but it also **increases test-time compute** substantially. We are running many forward passes: at each step expanding multiple thoughts and evaluating them. This is viable for short tasks (like 3-step solutions) or when using a smaller model for search guidance, but it might become expensive for very deep trees. In practice, one can adjust the branching factor B and depth or use optimizations like Monte Carlo Tree Search (where you selectively sample branches to explore more deeply based on promising outcomes). The **Long et al. (2023)** work referenced in the prompt engineering guide introduced a *ToT Controller* trained via reinforcement learning to more efficiently manage the search, as opposed to brute-force DFS/BFS [76] [77]. The controller learns when to expand or prune, improving efficiency – analogous to AlphaGo's learned search policy vs. naive tree search.

In summary, Tree-of-Thoughts is a powerful concept that generalizes chain-of-thought to multiple possibilities. It embodies the principle of **systematic exploration** in LLM inference, leveraging the model's strengths to both generate and judge ideas. ToT is especially useful when a task has many possible solution paths or when the correct solution requires avoiding deceptive local optima. It does, however, require careful prompt design (to get the model to propose diverse thoughts and evaluate them accurately) and can be computationally heavy. Importantly, ToT showcases how classical search algorithms and modern LLMs can be combined – hinting at a future of **hybrid neuro-symbolic reasoning** at inference time.

Having covered CoT, ReAct, and ToT, which progressively increased the sophistication of prompting and inference, we now turn to the very recent advances exemplified by **OpenAI's o1 model and the DeepSeek-R1 model**. These systems incorporate many of the ideas discussed (multi-step reasoning, self-consistency, search, process supervision) into large-scale training and inference pipelines. They represent the state-of-the-art in reasoning as of 2024–2025 and point toward the integration of multi-agent and multi-path approaches in production LLMs.

5. Advanced Multi-Path Reasoning: OpenAI o1 (2024) and DeepSeek-R1 (2025)

OpenAI's **o1** model, introduced in late 2024, is described as *"a new series of AI models designed to spend more time thinking before they respond."* [7]. This succinctly captures the essence of scaling *test-time reasoning* as an alternative to (or in addition to) scaling model size. OpenAI o1 was trained with a large-scale reinforcement learning procedure specifically aimed at teaching the model *how to reason using a chain-of-thought*, rather than only learning the end-task mappings [78] [5]. In other words, o1 learned to internally generate and use CoT rationales as part of its decision-making. Importantly, these chains-of-thought are **hidden** from the user by default – the model "thinks in private" and then produces a final answer [79]. This is a form of *internal prompting*: the model is trained to create a scratchpad of thoughts that are not directly emitted unless asked. By not outputting the raw chain-of-thought, the model can be more free and verbose in its internal reasoning (including trying out thoughts that might be wrong initially, since the user won't see them) and then only share the polished result.

The o1 model achieved impressive feats: ranking in the 89th percentile on CodeForces programming problems, scoring among top high school students on the AIME math competition, and even exceeding the performance of human PhDs on a graduate-level science QA benchmark [80] [81]. Much of this success is attributed to two things: *train-time compute* (using RL with human or AI feedback to instill better reasoning strategies) and *test-time compute* (allowing the model to use more steps and samples when answering) [78] [5]. Notably, OpenAI reported that o1's performance **smoothly improves** with more thinking time: if you allow it more steps or multiple attempts, it gets better [78]. This suggests diminishing returns from parameter count can be counteracted by increasing inference computation.

To illustrate o1's use of multi-path reasoning, consider its performance on the AIME 2024 math exam. With just a single attempt per problem, o1 solved about 74% of the questions [8]. When allowed to use **64 reasoning samples with a consensus vote**, this jumped to ~83% [8]. And when they went even further – generating 1000 candidate solutions and using a learned re-ranking model to pick the best – o1 reached 93% (near human-perfect) [8] [6]. This is a concrete example of scaling up test-time compute: from 1 to 64 to 1000 tries, each additional compute yielded better accuracy, eventually surpassing even what a single human expert might achieve on that test. The **shaded regions in OpenAI's charts** represent these majority

vote improvements, clearly showing how additional samples (and hence more reasoning trajectories) push performance upward [82] [83].

OpenAI o1 also employs a form of **multi-agent debate/verification** internally. Although full details aren't public, the "GPT-4o" versus "o1" comparison suggests that o1 (the RL-trained model) learned to use its chain-of-thought to self-verify and correct mistakes that GPT-4o (the baseline model without that specialized training) would miss [84] [85]. Anecdotal examples in OpenAI's blog show o1 breaking down a problem, catching an error in an intermediate calculation, and fixing it – behaviors encouraged by process-level feedback during training [84]. We can think of o1 as essentially GPT-4 with an *upgraded reasoning module*: it not only knows facts and how to produce text, but also how to systematically attack problems, when to try alternative approaches, and how to introspect on its own outputs. It's the closest realization so far of an "AI reasoner" that approaches problems like a human expert might (albeit still within the limits of pattern learning and not a guarantee of infallibility).

On the other side, **DeepSeek-R1** (released in 2025 by the company DeepSeek-AI) represents an open-source counterpart focusing on advanced reasoning. DeepSeek-R1's claim to fame is that it achieved performance comparable to OpenAI o1 on math, coding, and reasoning benchmarks, despite being developed outside the major labs [86]. The approach taken by DeepSeek is revealing: they started with a base 70B model (called DeepSeek-V3-Base) and applied **pure reinforcement learning** – notably *without any supervised fine-tuning on human chains-of-thought initially* [86] [87]. Through large-scale RL (likely using reward signals for correctness and reasoning quality), they obtained **DeepSeek-R1-Zero**, a model that "naturally emerged with numerous powerful and interesting reasoning behaviors" [88] [87]. These behaviors included *self-verification* (the model checking its answers), *reflection* (recognizing when it might be wrong and trying another approach), and the ability to generate very long, coherent CoT solutions [89] [59] – all without having been explicitly shown human-written solutions. This is a remarkable result: it suggests that, given the right training setup, an LLM can *learn to reason by itself* through trial-and-error reinforcement learning, developing strategies that we usually hand-craft via prompting.

However, the R1-Zero model had issues like sometimes rambling or getting stuck (endless loops, or mixing languages) [90]. DeepSeek addressed these by incorporating a **"cold-start" supervised phase**: they collected a set of high-quality reasoning trajectories (perhaps by filtering R1-Zero's outputs or using some synthetic data) and fine-tuned the model on those before further RL training [91] [92]. The final **DeepSeek-R1** model thus combines both supervised and RL training, and it *"achieves performance comparable to OpenAI-o1 across math, code, and reasoning tasks."* [91] [93]. Impressively, DeepSeek open-sourced not only R1 (which is a massive model with 670B "virtual" parameters, though internally uses sparsity to activate ~37B per token) [94], but also a suite of **distilled smaller models** (down to 1.5B) that still retained strong reasoning ability [95] [96]. For example, their 32B distilled model outperformed "OpenAI-o1-mini" (presumably a smaller version of o1) on various benchmarks [95]. This distillation was done by using R1 to generate a large set of reasoning examples and fine-tuning smaller models on them, demonstrating a path to make advanced reasoning more accessible without enormous compute costs for inference [97] [98].

DeepSeek-R1's training pipeline is notable for integrating **multi-stage reasoning enhancements**. They describe two RL stages (one to bootstrap reasoning patterns, another to align to preferences) and two supervised stages (to initialize some reasoning capability and to solidify non-reasoning knowledge) [99] [100]. They also emphasize *rejection sampling and fine-tuning* – likely generating many solutions and fine-tuning on the best – as well as a final **self-evolution loop** where the model iteratively improved itself [101] [102]. The result, R1, exhibits sophisticated behaviors like generating multiple solution paths and choosing the most

confident one (akin to self-consistency) and "reflection mechanism" mini-steps where it might question its own intermediate conclusions [103] [104]. In essence, R1 learned a kind of *multi-path prompting internally*: it can reason through different possibilities (multi-path) and even simulate multiple agents or perspectives in its own head (multi-agent), then consolidate the findings.

Both OpenAI o1 and DeepSeek-R1 underscore the theme that **scaling model *thinking* is the next frontier**. Instead of just making the neural network larger, we give it the ability to *use itself more effectively* by allocating more compute per query and by orchestrating its reasoning like a team of sub-agents or a search algorithm. These models blur the line between architecture and inference procedure – they are trained with objectives that explicitly encourage multi-step, self-checked reasoning. The gap between training and inference also narrows: whereas older approaches might train a model and then at inference time bolt on an algorithm like tree search, o1 and R1 show that we can train the model *to integrate those algorithms into its own operation*. For instance, R1's raw generation likely already includes reflective and verifying moves, and o1's hidden chains might be implicitly doing something like debating pros and cons of answers.

From a user perspective, these models still interface through prompts, but one can now **prompt at a higher level**. Rather than meticulously writing a chain-of-thought prompt, a user might simply ask a question and rely on the model's internal reasoning faculties to do something smart (maybe with a simple cue like "Think carefully"). However, understanding and sometimes guiding these internal processes can still improve outcomes – which leads us to consider how practitioners can leverage prompt engineering to elicit the best reasoning behavior.

In the next section, we turn to a practical "lab" where we look at *how to apply these reasoning techniques* in real scenarios. We will walk through prompt examples and even code snippets for using chain-of-thought, ReAct style prompts with tools, and Tree-of-Thoughts strategies. The goal is to give you hands-on intuition for improving LLM reasoning on your own tasks, using the concepts covered so far.

6. Scaling Test-Time Compute: Strategies for Better Inference

Before diving into the prompt engineering exercises, it's helpful to summarize and generalize the strategies for scaling up reasoning at inference time. Many of the methods we discussed can be seen as ways to spend more compute during inference in order to get more accurate or robust answers (as opposed to spending compute only on training or scaling model size). Here are several key strategies:

- **Multiple Samples and Ensemble (Bag-of-Answers):** One straightforward approach is to sample multiple outputs from the model and then combine them (e.g., by majority vote or a scoring function). This is essentially what self-consistency does for chain-of-thought [31] [34], and what OpenAI o1's 64-sample consensus does for answers [82] [83]. Even without full chain-of-thoughts, a simpler *Best-of-N* (BoN) strategy can be used: generate N answers and pick the one with the highest model-estimated likelihood or a separately trained answer ranker [35] [37]. Best-of-N can significantly outperform a single greedy output, as it mitigates randomness and lets the model explore alternative phrasings. However, if the model has a strong preference for a particular wrong answer (due to a learned bias), pure sampling may not help unless guided by a reward model.

- **Step-wise Voting or Beam Search:** Instead of sampling whole answers, one can sample at each step of a chain-of-thought (this was referred to as "Step-wise BoN") [105]. For example, at step 1 generate 5 possible thoughts, then at step 2 expand each of those with 5 continuations, etc.,

constructing a beam. This is essentially a beam search over reasoning sequences. Unlike tree-of-thoughts which might use model heuristics to prune, a straightforward beam search keeps the top K sequences by probability. This *guarantees* finding a high-probability solution (if one exists) but could miss a correct answer that has lower probability under the model. Thus, combining beam search with some form of re-ranking by correctness can be useful – for instance, generate a beam of candidate solutions and then use a verifier or solver to check which ones are actually correct. Beam search with a **learned scoring function** is what OpenAI did with o1 for AIME: 1000 samples re-ranked by a learned model gave the best result [8]. This learned model could be a fine-tuned version of the LLM to predict correctness or an external classifier. The general idea is to **separate generation and evaluation**: use the LM to propose many potential answers or chains, then use another mechanism (could be the same LM in a different mode) to choose the best.

- **Consistency Filtering and Cross-Checking:** Consistency can be enforced not just by majority vote, but by requiring that multiple independent reasoning paths arrive at not just the same final answer, but also consistent intermediate conclusions. For example, Uesato *et al.* (2022) proposed *verification through agreement*: the model tries to solve a problem in two different ways (say, forward reasoning and backward reasoning) and only trusts the answer if they match [106]. Another consistency filter is to have the model solve a problem *twice*, and if the answers differ, realize it is uncertain and perhaps then resort to a more careful approach or output a disclaimer. The forthcoming method "Agent Workflow" is a variant where the problem is broken into sub-problems and solved piecewise [107]. Ensuring internal consistency between the sub-answers is key; any inconsistency flags that the reasoning might have an error. These approaches effectively *use the model as its own consistency checker*, which is valuable because sometimes the model might not know absolutely if an answer is correct, but it can notice if two attempts conflict and thus signal low confidence.

- **Dynamic Routing and Tool Use:** Some questions might be best answered with a certain tool or a particular strategy. For instance, a arithmetic-heavy query might be routed to a "calculator" module, whereas a factual question might be routed to a "web search" module. This concept of **tool routing** can be implemented with a simple decision tree (if question matches a regex for math, do X, else do Y) or via a learned classifier that picks a prompting strategy. In prompt-engineering terms, one might have multiple prepared prompts: a CoT prompt for math, a ReAct prompt for knowledge queries, a code execution prompt for programming tasks, etc. A preceding step (perhaps an LLM itself) decides which prompt to use. This saves compute by not doing every strategy for every query, and it can improve accuracy by applying the right method for the right task. Think of it as an *ensemble of specialists* used at inference – sometimes called a **Mixture-of-Experts** at the prompt level. For example, Google's PALM-RL (2022) had separate "solver" and "checker" modules for math problems; the solver would propose an answer and the checker (a verifier LM) would approve or veto it [19] [108]. That is a form of routing (first to solver, then optionally to a checker if needed).

- **Depth Adaptive Computation:** Not every query needs a long chain-of-thought. A simple factual question ("What is the capital of France?") can be answered in one step confidently by most LLMs. But a multi-step puzzle requires more work. One strategy is to make the amount of computation **adaptive** to the problem difficulty. Early attempts at this include *self-terminating CoT*: the model might learn to output an intermediate conclusion like "Therefore, the answer is X" once it's confident, otherwise continue reasoning. More directly, one can have a heuristic that measures confusion or uncertainty (e.g., how much the model's token probabilities drop, or whether the model's answer is self-contradictory) and use that to decide if more reasoning is needed. The *ReAct* framework

naturally allows this: the model can keep taking actions until a solution is found. In a tree search, one can allocate more expansions to harder questions (like iterative deepening – try shallow reasoning first, if it fails, go deeper). The goal is to not waste computation on easy tasks while still having it available for hard tasks.

- **Beam Re-ranking and Value Functions:** This ties in with earlier points but specifically refers to using a secondary model or method to rank candidate outputs beyond simple majority vote. A common scenario is to train a **verifier** model that given a question and a proposed solution (with or without chain-of-thought) outputs a score of correctness. If you have such a verifier (it could be a smaller model or the same LLM prompted to "critique" the answer), you can generate many candidate answers and pick the one the verifier rates highest [63]. OpenAI's 1000-sample re-ranking essentially did this with a learned scoring function [6]. Another example is AlphaCode (DeepMind's coding model) which generated many programs and then executed and tested them to filter out only those passing all tests – a brute-force but effective inference-time selection mechanism. In LLM reasoning, we might not have executable tests, but we can sometimes construct a proxy: e.g., for a math proof, verify each step is logically valid; for a riddle, ensure the answer fits all clues. Whenever an automated checker can be applied, it should be used to filter or rerank outputs. If not, an ML-based verifier is the fallback.

It's worth mentioning the **Agent Workflow** and **Self-Refine** methods studied by Wu et al. (2024) in comparing to o1 [62]. Agent Workflow involves explicitly planning sub-questions and solving them – this is akin to an LLM breaking a problem into parts (sometimes called "tree-of-thought prompting" in another context) and is especially useful when different sub-parts require different tools or skills. Self-Refine, as noted, is having the model iteratively improve its answer by critique. This can be seen as adding another loop of compute: answer → review → answer → … until stable. Each refinement is extra inference cost but can yield a better final result.

The overarching theme is **trading compute for accuracy** at test time. All these methods consume more CPU/GPU cycles and more model calls, but they tend to deliver higher quality results, particularly on tasks where single-pass generation is unreliable. In practical deployment, one might implement a cascade: try the fastest method first (say, direct answer). If confidence is low or the stakes are high (e.g. an important query), then engage a slower, more thorough method (like CoT + self-consistency, or a tool-using agent). This way, simple questions get quick answers, and hard ones get heavy reasoning.

In the lab section that follows, we'll see some of these strategies in action through example prompts and experiments. Keep in mind that while adding more inference steps generally improves reasoning performance [5] [8], it can also expose the model to new failure modes (like hallucinating tool usage or creating convoluted but wrong explanations). Therefore, careful prompt design, constraints on actions, and use of verification steps remain important to maintain reliability.

7. Prompt-Engineering Lab: Exercises in Reasoning

In this section, we'll walk through a series of practical exercises to illustrate how to use prompting techniques – Chain-of-Thought, ReAct, and Tree-of-Thoughts – to improve reasoning on various tasks. We assume you have access to a powerful language model (such as GPT-4 or an open-source equivalent) via an API or interface. The exercises will be in prose, but you are encouraged to actually try them out with an LLM to see the outcomes.

Exercise 7.1: Improving Open-Domain QA with CoT and ReAct

Task: You have a question: *"The Seven Brief Lessons on Physics was written by an Italian physicist who has worked in France since what year?"* This is a multi-hop question (it asks for a year, and you need to know who wrote that book and then find since when that person has worked in France).

- **Step 1:** Try a **direct prompt** (no chain-of-thought) and see what you get:

```
Q: The "Seven Brief Lessons on Physics" was written by an Italian physicist
that has worked in France since what year?
A:
```

Expected Outcome: A model might not get this directly, or might guess incorrectly. It needs to figure out the author (Carlo Rovelli), then find the detail about working in France since 2000.

- **Step 2:** Now use a **Chain-of-Thought** approach:

```
Q: The "Seven Brief Lessons on Physics" was written by an Italian physicist
that has worked in France since what year?
Let's think step by step.

1. "Seven Brief Lessons on Physics" is a book; we need the author and a
year.
2. The book was written by Carlo Rovelli, an Italian physicist.
3. The question asks: since what year has he worked in France?
4. Carlo Rovelli has worked in France since 2000 (he moved to France and
has been there since year 2000).
Answer: 2000.
```

Try it: Many LLMs, when prompted with "Let's think step by step," will generate a reasoning chain somewhat like the above. Ideally, it concludes "since 2000." Check if it matches known facts (Carlo Rovelli indeed has been in France since 2000).

- **Step 3:** Use a **ReAct prompt** with a tool (e.g., a Wikipedia search function). If your interface supports plugins or you can simulate tool use:

```
Thought: "Seven Brief Lessons on Physics" author? It's likely Carlo
Rovelli. I'll search for Carlo Rovelli.
Action: Search["Carlo Rovelli Seven Brief Lessons on Physics"]
Observation: Carlo Rovelli... (it might show a snippet saying he's been
working in France since 2000).
Thought: The author is Carlo Rovelli, who has worked in France since 2000.
Action: Finish["He has worked in France since the year 2000."]
```

In practice, you might not literally write the Thought/Action/Observation yourself; instead you instruct the model in the prompt with an example of how to use a search API. If done correctly, the model will output something like the above, including performing the search action. This exercise demonstrates how using a tool can get the evidence needed.

Discussion: In open-domain QA, a major challenge is that the model might not recall a specific fact, but it can reason out what to search for. A pure CoT helps organize the multi-hop reasoning, but may still rely on memory. ReAct goes a step further by actively retrieving the needed fact. When you tried Step 2, if the model already knew Carlo Rovelli's details, it might answer directly. If not, Step 3 would show how it can use search to find the answer. Many QA systems (e.g. Bing's AI search or the ReAct paper's QA agent) use this approach of interleaving thoughts and tool use to great effect [41] [109].

Exercise 7.2: Solving a Math Word Problem with CoT and Self-Consistency

Task: Solve the following math problem, and use self-consistency to verify the result: "*Jenny had 48 apples. She gave an equal number of apples to 3 friends and had 6 left for herself. How many apples did each friend get?*"

- **Step 1:** CoT prompt:

```
Q: Jenny had 48 apples. She gave an equal number of apples to 3 friends and
had 6 left for herself. How many apples did each friend get?
Let's think step by step.
```

The model might produce: *"Jenny has 48 apples in total. She kept 6, so she gave out 48 - 6 = 42 apples to friends. These 42 apples were given equally to 3 friends, so each friend got 42 / 3 = 14 apples. Therefore, each friend got 14 apples."*

This is a straightforward problem and a single CoT likely gets it right: **14**.

- **Step 2:** Now apply **self-consistency**. Instead of one chain, sample multiple chains. With an API, you could do something like:

```
import random
results = []
for i in range(5):
    response = model.complete(prompt_with_CoT, temperature=0.7,
seed=random.randint(0,10000))
    answer = extract_answer(response)
    results.append(answer)
print(results)
```

The expectation is all chains will come out with "14" in this case, since the problem is well-defined. If you get unanimity (e.g., `[14, 14, 14, 14, 14]`), you are confident the answer is 14. If there was a discrepancy, you'd take the majority. For a more complex problem, you might see occasionally a chain with a wrong path. Self-consistency helps double-check.

- **Step 3 (Optional):** Try a harder arithmetic or algebra problem where the model is more prone to mistakes and see how self-consistency improves accuracy. For example: *"A store had 120 batteries. It sold 1/3 of them on Monday and 1/4 of the remainder on Tuesday. How many batteries are left?"* A single CoT might slip up if not careful, but sampling multiple times may yield the correct answer more reliably and you can pick by majority.

Discussion: This exercise shows how to programmatically implement self-consistency. In many notebook environments or APIs, you can specify `n` (number of completions) or simply loop with different random seeds as above. The `extract_answer` function would parse the model's output string to get the final numeric answer (perhaps by finding the last number mentioned). Self-consistency is especially useful in cases where the model's reasoning is non-deterministic and a single pass might latch onto an incorrect heuristic. By aggregating multiple tries, you harness the model's full distribution of reasoning patterns [31] [34]. If most of those patterns point to the same answer, it's likely correct.

Exercise 7.3: Implementing a Simple Tree-of-Thoughts for a Puzzle

Task: Use a basic Tree-of-Thoughts approach to solve a small search problem. A classic example is the 8-puzzle or a simpler game like the "24 game" we discussed. Here, let's do a number puzzle: "*You have the numbers 2, 3, 4, 5. Insert arithmetic operations (+, -, ×, ÷) between them (in some order) to achieve the result 24.*"

This is the Game of 24 (with a specific set). Let's apply a limited search:

- **Step 1:** Brainstorm possible first steps (the model can generate these). For instance, pair up numbers or pick an operation to do first. One possible thought: combine 5 and 4 first (5×4=20), then you have 20, 3, 2 to make 24. Another thought: maybe 5 + 3 = 8, etc.

Using prompting: *"We need to reach 24 from 2,3,4,5. Possible first operations: (5×4)=20, or (5+4)=9, or (5-4)=1, or (5÷4)=1.25, etc. Let's try one."* This is a bit advanced to do purely by prompting, as it requires controlling the search. Instead, consider doing it programmatically with an LLM in the loop:

- At state = `[2,3,4,5]`, ask the model to suggest one operation to apply (like "Pick two numbers and an operation"). Get several suggestions: e.g., it suggests (4×5=20), (3+2=5), etc.
- For each suggestion, apply it and reduce the state (e.g., [2,3,4,5] + (4×5) → new state [2,3,20]). Now ask the model from state [2,3,20] how to reach 24.

This can be done with a structured prompt where the model knows it's in a search:

```
Numbers: [2, 3, 4, 5], Target: 24.
Step 1: Try an operation.
```

Model might output: "Try 5×4 = 20. New set: [2, 3, 20]." Then you feed back: "Numbers: [2, 3, 20], Target: 24. Step 2: Try an operation." Model: "Try 20 + 2 = 22. New set: [22, 3]." Next: "Numbers: [3, 22], Target: 24. Step 3: Try an operation." Model: "Try 22 + 3 = 25. New set: [25]. (This overshot, not 24.)" – at this point it failed that path.

Instead, if model had tried another approach (like from [2,3,20], do 20 + 3 = 23, then 23 + 1? doesn't work; or 20 + 4... etc.), it might realize a misstep.

This exercise is a bit complex to do fully manually, but it illustrates how you might use an LLM to explore branches. Alternatively, one could manually enumerate combinations and ask the model to evaluate.

- **Alternate approach:** Use the **ToT prompting technique** (Hulbert 2023) which encapsulates a short search in one prompt. For example:

  ```
  Solve 2,3,4,5 -> 24. Imagine three different experts are thinking:
  Expert1: (5 × 4) = 20; Now with 20,2,3 -> 20 + 2 + 3 = 25 (too high).
  Expert2: (5 × 3) = 15; Now 15,4,2 -> 15 + 4 + 2 = 21 (not 24).
  Expert3: (5 - 2) = 3; Now 3,3,4 -> 3 × 4 + 3 = 15 (not 24).
  Evaluate these: None reached 24.
  Now experts try second step adjustments:
  Expert1: 20 + 4/2 = 22 (no),
  Expert2: 15 + (4 × 2) = 23 (no),
  Expert3: (5 - 2) = 3; 3 + 3 = 6; 6 × 4 = 24 (got it!).
  So one solution is: (5 - 2 + 3) × 4 = 24.
  ```

  This shows a possible chain where the third expert found a solution on a revised attempt. Indeed, $(5 - 2 + 3) \times 4 = 24$.

Discussion: Tree-of-Thoughts is harder to emulate without coding, but this exercise gives a flavor. The key is to generate intermediate possibilities and evaluate. Even if done manually, the model could help by identifying which partial sums or products seem promising (the evaluation heuristic). For instance, it might label partial result 22 as "maybe close but need 2 more" and partial 15 as "far from 24".

From this lab, the **takeaways** are:

- By explicitly prompting intermediate steps or tool usage, we can solve queries that base LLM responses would miss.
- Combining the model's outputs (via majority or other criteria) yields more reliable answers for multi-step problems.
- We can harness the model in a loop (repeated querying) to simulate search or iterative refinement, albeit with some overhead in prompt engineering.
- Prompt engineering for reasoning often involves giving the model a structured role (like an expert solving step by step, or multiple experts debating). This helps the model adhere to a logical process rather than jumping to an answer.

8. Benchmarks: Performance vs. Compute Trade-offs

It's one thing to qualitatively describe these techniques, but quantitative benchmark results help illustrate their impact and the trade-offs involved. Here we summarize some key findings from research, highlighting

how reasoning accuracy improves with increased test-time compute, and where diminishing returns or practical limits arise.

- **Arithmetic/Math Benchmarks:** On GSM8K (grade-school math), few-shot chain-of-thought prompted models made huge leaps. Wei et al. reported GPT-3 175B jumped from *~18% accuracy to ~40%* by using CoT prompting [34]. Further, using self-consistency decoding, accuracy climbed to **58.1%** [34]. More recently, GPT-4 reportedly exceeds 90% on GSM8K, and OpenAI's o1 (with reasoning training) essentially saturates it [110] [8]. As a result, researchers moved to harder math tests like MATH (competition problems) and AIME. On the 2024 AIME, as discussed, GPT-4 with no special inference only got 12% of problems, whereas o1 reached 74% with one try and 83% with 64-sample consensus [8]. Pushing test-time compute even further (1000 samples + rerank) gave 93% [8]. However, note the *diminishing returns*: going from 1 to 64 samples gave +9% absolute, but going from 64 to 1000 (with a smart reranker) gave another +10%. Each additional gain required an order of magnitude more samples. There is a practical limit: not every scenario allows 1000 tries; the latency and cost could be high. But if accuracy is paramount (e.g., an AI exam-taking system), that extra compute can beat human-level performance in narrow domains [6].

- **Commonsense and Logical Reasoning:** Benchmarks like Big-Bench tasks, StrategyQA, and ARC (challenge set) benefit from CoT prompting as well. Self-consistency improved StrategyQA accuracy by ~6.4 points (from ~70% to ~76% on a certain model) [32]. OpenAI's o1 was measured on MMLU (a collection of academic exam questions across subjects). It improved over a GPT-4 baseline on 54 out of 57 subjects [111] [112]. In some categories, the gains were modest, but in others, substantial – especially those requiring step-by-step thinking (like math word problems or nuanced logical inference). These improvements often come with some extra inference compute (o1 was evaluated "at maximal test-time compute" meaning it likely used multiple attempts per question) [113] [114]. A notable result: o1 *with vision* (for multimodal tasks) scored 78.2% on a visual reasoning test (MMMU), marking the first time an AI matched human performance there [115] [112]. This suggests the reasoning training generalized even to multimodal data.

- **Interactive Benchmarks:** For ReAct-style agents, success is measured differently (e.g., success rate in a game or task completion). Yao et al. reported on HotpotQA (multi-hop QA): ReAct achieved higher QA accuracy than a chain-of-thought baseline (exact figures might be like mid-80s% vs high-70s%, though exact numbers are not in the snippet) [39]. In the ALFWorld virtual environment, a ReAct agent had a success rate of 34% vs much lower for pure RL agents [46]. WebShop (a simulated shopping task) saw a 10% absolute success improvement with ReAct over prior methods [46]. These improvements are *without additional training data* – they arise from better prompting and allowing the model to use more steps. However, each step is an API call or model inference, so a trade-off: ReAct might take 10–20 turns (model calls) to answer a single question that a single-turn model might try (and fail) to answer in one go. The computation is higher, but the payoff is success on tasks that were previously out of reach for static prompts.

- **Process Supervision Effects:** How does process supervision (training with step-level feedback) reflect in metrics? In OpenAI's math paper [11], the process-supervised model solved 78% of a Math dataset subset, versus an outcome-supervised one solving much fewer (the exact comparison was not given, but presumably outcome-only was significantly lower, perhaps ~50-60%). This shows training for reasoning can boost final performance too – it's not just an inference trick, but a better model. Interestingly, process-supervised models tend to produce *more accurate chains-of-thought at*

inference. If you inspect their outputs, they make fewer logic leaps and each step is more likely correct, as the reward model has taught them to do so. This can reduce the need for huge ensembles at inference because the single chain is more trustworthy. That said, even process-supervised models still benefit from self-consistency and related methods, just starting from a higher base.

- **Scaling Curves:** A general trend observed is that as you allow more **compute (steps or samples)**, smaller models can sometimes catch up to larger models that are limited to less compute. For example, a 30B model with CoT and self-consistency might rival a 100B model with direct prompting on certain tasks [34]. This is promising for those who cannot afford the largest models but can afford some extra processing. Of course, the largest models with these techniques perform best of all. The OpenAI o1 blog included a remark: *"The constraints on scaling this approach differ from those of LLM pretraining"* [78] [5] – meaning to improve o1, you consider how much RL steps and inference time to give it, not just how many parameters or tokens to train on. It's a new paradigm of scaling along a different axis.

- **Limitations and Failure Modes:** Even with advanced reasoning, models can err. Sometimes they converge confidently on a wrong answer (especially if the majority of reasoning paths share a flawed assumption). Self-consistency can fail if the model has a systemic bias: e.g., if 8 out of 10 chains all make the same mistake, the vote will be wrong (just confidently so). Tree-of-thoughts and searches can blow up combinatorially if not pruned – there are tasks where the branching factor is too high for brute force. Researchers have noted that for tasks requiring dozens of steps (like complex code synthesis or long proofs), current models still struggle due to accumulating errors or the sheer difficulty of long-horizon credit assignment. Process supervision helps but isn't a solved issue in very long chains.

From a **compute perspective**, each of these techniques requires more FLOPs per query. There is an open challenge: *Can we get the benefits of reasoning without linearly increasing cost with complexity?* Ideas like **adaptive computation** and learned controllers (as in ToT with RL) are attempts to use compute more efficiently: spend it where needed, skip it where not. Another challenge is **calibration**: knowing *when* the model's answer is trustworthy. Often, self-consistency helps calibrate (if all samples agree, high confidence; if they diverge, low confidence) [8]. But sometimes models can be consistently wrong (particularly if they all share the same flaw). Developing complementary checks – like external verifiers or using domain knowledge – is an ongoing area.

In practical terms, benchmark results encourage us that if we are willing to tolerate, say, a 5x increase in latency, we can often cut the error rate significantly. If we tolerate a 50x increase, we might approach very high reliability on some tasks. The right balance will depend on the application. For a mission-critical calculation, 100x more compute to be sure might be fine. For a casual chatbot query, even 2x latency might hurt UX.

Finally, it's notable that **human-level performance** has been reached or surpassed in many narrow benchmarks (math contests, trivia quizzes, etc.) by combining large models with reasoning techniques [8] [115]. However, true reasoning ability in the general sense is far from solved – these models can still make silly mistakes outside their training distribution or be fooled by adversarial logic puzzles. Benchmarks evolve: once one is solved, researchers propose new, harder ones (for example, ever more complex puzzle

collections, or requiring reasoning about novel concepts). The cat-and-mouse game continues, but the methods in this guide give us a strong toolkit for tackling new reasoning challenges.

9. Summary, Key Insights, and Open Challenges

Summary of Techniques: We began by introducing the motivation for enhanced reasoning in LLMs – to handle tasks that single-shot answers cannot, and to do so in a compute-efficient way by leveraging inference-time algorithms. We saw how **Chain-of-Thought prompting** unlocks step-by-step reasoning and significantly improves performance on complex tasks by having the model *explain before answering* [1] [15]. Building on that, **self-consistency decoding** was introduced as a way to sample multiple reasoning paths and choose the most consistent answer, leading to large accuracy gains across math and reasoning benchmarks [31] [34]. We then explored **ReAct**, which merges reasoning with tool use or actions in an environment, allowing models to dynamically fetch information and interact with problems rather than solve them in a vacuum [39] [40]. This required a structured prompting approach (interleaving Thoughts and Actions) and showed clear benefits in interpretability and capability (e.g., solving multi-hop questions, games, etc., that were previously out of reach) [39] [46].

We also delved into **process supervision**, highlighting that training or rewarding intermediate steps can yield more reliable and aligned reasoning processes [11] [51]. Process supervision was a common thread in advanced models like OpenAI's *o1* (trained via RL to refine its chain-of-thought) and DeepSeek's *R1* (trained via RL from scratch to develop reasoning behaviors) [78] [86]. These models illustrate the cutting edge: they integrate multi-path reasoning natively, using techniques akin to self-consistency, debate, and search *within* their inference. The **Tree-of-Thoughts** framework generalized the idea of exploring multiple reasoning branches, employing search algorithms guided by the model's own evaluations [69] [60]. We saw how ToT can dramatically improve problem-solving on tasks requiring exploration, essentially by finding correct solutions that a linear CoT would miss [72] [73].

Key Insights:

1. **Reasoning is an emergent property that can be amplified by prompting.** Large LMs have latent reasoning capabilities that only manifest when prompted to use them (e.g., through CoT). At sufficient scale, even a simple phrase like "Let's think step by step" triggers complex behavior [22]. This means a lot of "intelligence" in LMs is not about stored facts, but about on-the-fly computation that we can coax out with the right cues.

2. **Inference-time computation is a new axis for performance.** We can trade more inference steps for higher accuracy, similar to how one might trade training data or model size [78]. This opens up flexibility: rather than retraining a model for every task, we can often just prompt it better or run an algorithm around it (like majority voting) to get the needed performance [1] [8]. It's like using the model as an engine and building various cars around it for different terrains.

3. **Interactivity and external tools immensely expand an LLM's scope.** By using actions (ReAct) or tool APIs, models overcome one of their main limitations: fixed knowledge and inability to perform explicit calculations. The synergy of "reason to decide an action" and "action to inform reasoning" is powerful [41] [109]. This transforms static QA models into agents that can, for instance, use a calculator, query a database, or control a simulated world.

4. **Quality of reasoning can be improved by feedback at the step level.** We saw that process supervision (via human feedback or reward models) can drastically reduce errors like hallucinations or logical slips in chains-of-thought [11] [54]. This has big implications for alignment: a model that is rewarded for each correct step is less likely to produce a convincing but wrong final answer, since it had to stay truthful throughout the chain. It also yields more understandable solutions – a critical aspect if AI is to work alongside humans.

5. **Combining learned models with search or programmatic frameworks yields best results.** The Tree-of-Thoughts approach and others like it demonstrate that classical search algorithms complement LLMs well [69] [70]. The LM provides the intuition (possible moves, heuristic evaluations) and the search provides rigor (systematically covering possibilities). Neither alone would be as powerful: pure search would be intractable in language spaces, and pure intuition might miss solutions or get stuck. This combination is reminiscent of how humans solve problems: a mix of intuition and deliberate checking of options.

Open Challenges:

Despite the progress, there are many challenges and research questions remaining:

- **Scale vs. Efficiency:** Using 1000 inference samples or extremely long chains-of-thought is not always practical. How can we make models that are *both* smart and fast? There is ongoing work on *sparse computation* (activating only parts of a network as needed) and *early exiting* (dynamically shortening or lengthening the reasoning). Possibly, future models could internally decide to only use heavy reasoning on hard parts of a problem.

- **Generality of reasoning:** A model might do well on benchmarks it was tacitly exposed to (via fine-tuning or RL training) but may struggle on truly novel problems requiring creativity or deep understanding (for example, writing a short mathematical proof that requires an “aha!” insight). We need to test models on more **open-ended reasoning tasks**. Some efforts like the BIG-Bench suite and “open-ended question answering” try to push this, but it’s hard to measure. It’s an open challenge to have a model *invent* a new solution method on the fly if all known methods fail.

- **Reliability and Verification:** Even with process supervision, models can make subtle mistakes that go unnoticed. In a long chain, a single arithmetic error can derail the final answer. Humans can review each step, but that’s not scalable. Developing automated verifiers for intermediate steps (like math checkers, logic proof verifiers, etc.) is crucial. Some work, as we saw, is blending model output with external checkers (e.g., executing code or formal proofs). The ultimate dream is a **fully self-checking model**: one that not only generates reasoning but internally verifies each inference against some ground-truth mechanism (like a theorem prover or a Python interpreter). We are not there yet for general language reasoning.

- **Prompt Robustness:** Prompt engineering is powerful but also ad-hoc. Small changes in prompt wording can affect results. An open problem is how to make reasoning **robust to prompt variations**. Techniques like few-shot prompting help by giving exemplars, but choosing those exemplars is an art. Could we have the model itself suggest how it should be prompted? Or automatically learn a prompt that elicits reasoning for a class of problems? There are meta-learning ideas (letting the model figure out the best prompting style) that are still being explored.

- **Multi-agent collaboration:** We touched on multi-agent debate (where two models discuss) and workflows splitting tasks. Extending this, one could imagine an ensemble of specialized models (like different "experts": a mathematician, a historian, a programmer) that collaborate on a single complex query. How to coordinate them is an open question – it could be via a controlling model or via a shared workspace they read/write (like a scratchpad that multiple models edit). This resembles how human teams solve problems. Early experiments (e.g., using two ChatGPT instances to debate a question) show some promise but also highlight issues like the models reinforcing each other's wrong ideas. Designing interaction protocols that yield true "wisdom of crowds" effects is challenging.

- **User Interaction:** In a deployed setting, a user could be part of the reasoning loop. For instance, a chatbot might ask the user for clarification or for a piece of data ("I need to look up your schedule – do I have permission?"). This is a form of process supervision (the user guides the process) and tool use (the model asks the user to act as a tool). Figuring out how LLMs can effectively query users for the right info, or explain their reasoning in a user-comprehensible way without overwhelming them, is an open UX problem. It ties into trust: users need to trust the system's reasoning, which suggests the system should sometimes show its chain-of-thought (when appropriate) in a simplified manner.

- **Evaluation of reasoning quality:** Traditional accuracy metrics don't fully capture the quality of reasoning. Two models might get the right answer, but one did so by luck or brute force, while the other had a correct reasoning. If the task is just QA, both score 1.0 for that question. But for safety and progress, we care *how* they got it. Evaluating chains-of-thought is tricky – it might require human judgment. Automated metrics (like "% of steps that are logically valid") are domain-specific. As more models use hidden CoT (like o1), how do we ensure they reason correctly internally? There's an open area of research in *interpreting and auditing* the internal reasoning of models, perhaps by sampling their hidden thoughts or by stress-testing with known pitfalls.

In conclusion, the field is moving toward **making LLMs not just knowledge bases but active problem solvers**. Prompting techniques and inference algorithms form the toolbox for this transformation. We've seen that a combination of these can push AI systems to superhuman performance on certain benchmarks, yet the ultimate goal of general, reliable reasoning AI is still on the horizon. As a practitioner or researcher, one should keep a balance of optimism and skepticism: these tools are powerful, but each has failure modes that need care. By continuing to refine process supervision, integrating search and learning, and rigorously evaluating reasoning, we will get closer to AI that can truly *think through* the hardest problems – perhaps one day matching the flexibility and depth of human reasoning, or even exceeding it in creative ways.

Further Exploration: To deepen your understanding, consider reading the reference papers in detail (Wei et al. 2022 [1] for CoT, Wang et al. 2023 [29] for self-consistency, Yao et al. 2023 [39] for ReAct, OpenAI 2024 [78] on o1, DeepSeek 2025 [86] on R1, and Yao et al. 2023 [66] for Tree-of-Thoughts). Try replicating small-scale versions of their experiments with open-source models (like using HuggingFace Transformers). And if you're inclined, tackle some open challenges: for example, design a prompt that makes a model check each step of its reasoning twice, or build a simple verifier that catches arithmetic errors in a chain-of-thought. The lab ideas we discussed can be expanded into larger projects or research investigations. Happy prompting and reasoning!

Study Questions: 1. Why does chain-of-thought prompting often fail with small models but succeed with very large ones? Discuss the role of model scale in emergent reasoning abilities [14] [9]. 2. Describe a scenario where self-consistency would not improve accuracy, even if you sample many reasoning paths. What does this reveal about the model's errors or biases? 3. ReAct brings tool use into the picture. List three different "tools" an LLM could use and how it would benefit reasoning (e.g., a calculator for math, a Python interpreter for code, a knowledge base for facts). 4. In process supervision, what are the pros and cons of training a reward model on human-labeled steps versus using automated feedback (like unit tests for code, or equation checks for math)? [116] [51] 5. Explain the Tree-of-Thoughts approach in your own words. How is it different from simply prompting "Let's think step by step"? [69] [60] 6. OpenAI's o1 model improved with more "thinking time". How would you quantitatively determine the optimal number of samples or reasoning depth to use for a new task, balancing accuracy and cost? (Hint: consider diminishing returns and maybe an adaptive strategy). 7. If you have a model that tends to hallucinate facts, which techniques from this guide would you apply to mitigate that? Provide a brief plan. 8. Imagine you could design a future benchmark for reasoning. What would it involve to really test an AI's reasoning (and not just its training data memorization)? Describe one or two sample tasks from that hypothetical benchmark.

By reflecting on these questions and experimenting with the techniques in practice, you will strengthen your mastery of prompting, test-time compute, and process supervision for reasoning in language models. The field is rapidly evolving – but the core ideas of breaking problems into steps, exploring alternatives, and rigorously supervising the process will remain central to building more intelligent and reliable AI systems [18] [5].

---

[1] [2] [3] [4] [9] [10] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [108] [2201.11903] Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
https://ar5iv.labs.arxiv.org/html/2201.11903

[5] [6] [8] [78] [80] [81] [82] [83] [84] [85] [110] [111] [112] [113] [114] [115] Learning to reason with LLMs | OpenAI
https://openai.com/index/learning-to-reason-with-llms/

[7] OpenAI o1 Hub | OpenAI
https://openai.com/o1/

[11] [13] [53] [54] [58] [106] [116] cdn.openai.com
https://cdn.openai.com/improving-mathematical-reasoning-with-process-supervision/Lets_Verify_Step_by_Step.pdf

[12] [51] [52] [55] [56] [57] aclanthology.org
https://aclanthology.org/2025.findings-acl.747.pdf

[29] [30] [31] [32] [33] [34] [2203.11171] Self-Consistency Improves Chain of Thought Reasoning in Language Models
https://arxiv.org/abs/2203.11171

[35] [36] [37] [62] [63] [105] [107] [2410.13639] A Comparative Study on Reasoning Patterns of OpenAI's o1 Model
https://arxiv.org/abs/2410.13639

[38] [39] [40] [41] [42] [43] [44] [47] [109] ReAct: Synergizing Reasoning and Acting in Language Models
https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/

[45] [46] [48] [49] [50] arxiv.org
https://arxiv.org/pdf/2210.03629

59 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 deepseek-ai/DeepSeek-R1 · Hugging Face
https://huggingface.co/deepseek-ai/DeepSeek-R1

60 61 67 68 69 70 75 76 77 Tree of Thoughts (ToT) | Prompt Engineering Guide
https://www.promptingguide.ai/techniques/tot

64 65 66 72 73 74 [2305.10601] Tree of Thoughts: Deliberate Problem Solving with Large Language Models
https://arxiv.org/abs/2305.10601

71 promptingguide.ai
https://www.promptingguide.ai/_next/image?url=%2F_next%2Fstatic%2Fmedia%2FTOT.3b13bc5e.png&w=3840&q=75

79 Learning to Reason with LLMs - Simons Institute
https://simons.berkeley.edu/news/learning-reason-llms

101 102 arxiv.org
https://arxiv.org/pdf/2501.12948

103 104 Marco-o1: Towards Open Reasoning Models for Open-Ended Solutions
https://arxiv.org/html/2411.14405v1